

Exp No.: 7 Endorsement Policy Configuration in Hyperledger Fabric

Aim

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

An Endorsement Policy defines which organizations must approve (endorse) a transaction before it can be committed to the ledger. Hyperledger Fabric uses endorsement policies to ensure that transactions are validated according to predefined business rules.

For example, `OR('Org1MSP.peer', 'Org2MSP.peer')` means that either Org1 or Org2 can endorse the transaction. Similarly, `AND('Org1MSP.peer', 'Org2MSP.peer')` means that both organizations must endorse the transaction before it becomes valid. Endorsement policies improve security, trust, and transaction integrity in permissioned blockchain networks.

Procedure

Step 1: Navigate to the Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

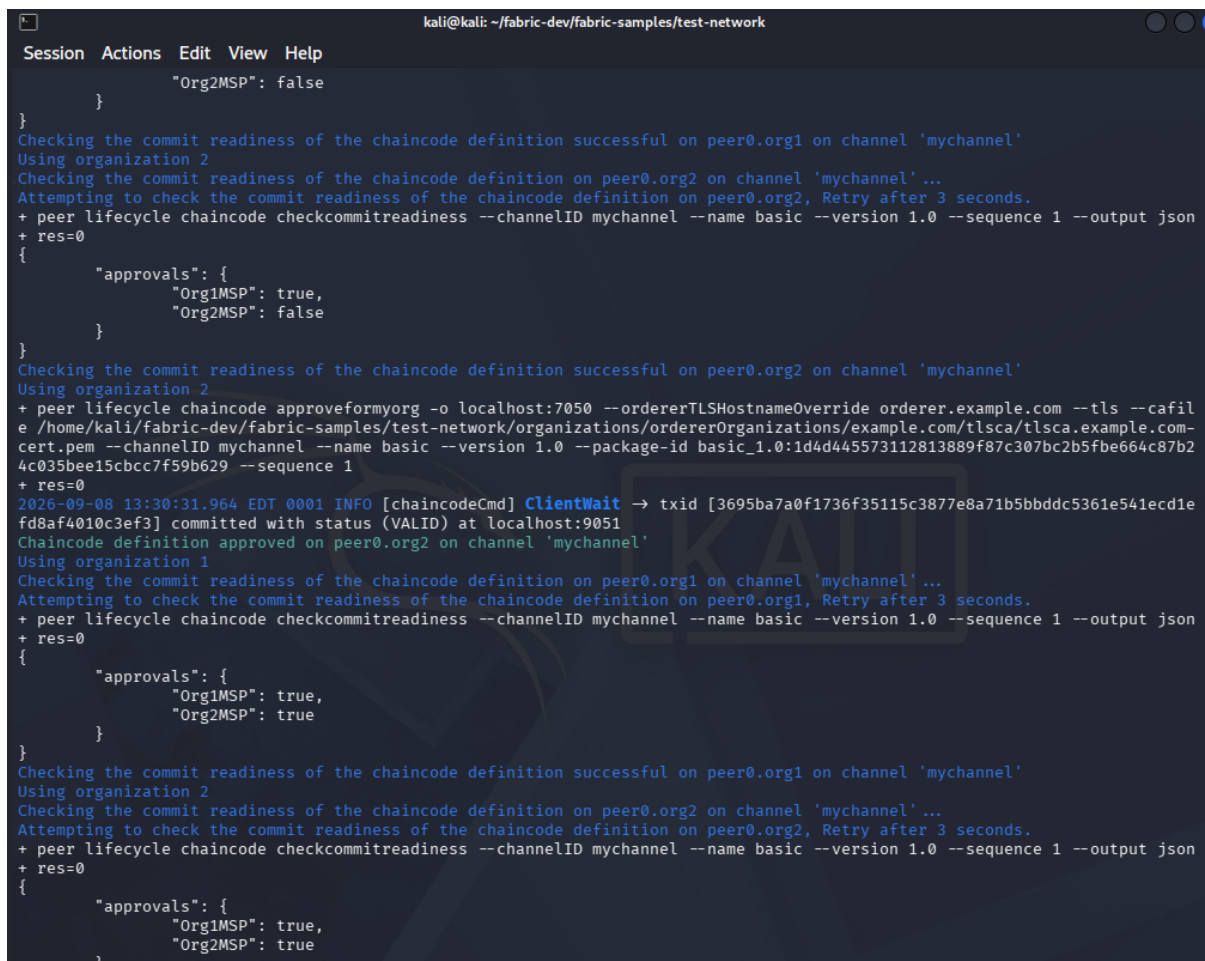
Step 3: Deploy Chaincode with OR Endorsement Policy

Deploy the Asset Transfer chaincode with a policy allowing either organization to endorse.

```
./network.sh deployCC -ccl javascript -ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccep "OR('Org1MSP.peer', 'Org2MSP.peer')"
```

Step 4: Verify Chaincode Commitment and Approvals

Check that both organizations have approved and the chaincode is committed.

A terminal window titled 'kali@kali: ~/fabric-dev/fabric-samples/test-network' showing the execution of Fabric network commands. The output displays the approval status for 'Org1MSP' (true) and 'Org2MSP' (false), followed by a successful commit readiness check on peer0.org1. It then shows the approval status for 'Org1MSP' (true) and 'Org2MSP' (false), followed by a successful commit readiness check on peer0.org2. The chaincode is then committed with status (VALID) at localhost:9051. Finally, it shows the approval status for 'Org1MSP' (true) and 'Org2MSP' (true), followed by a successful commit readiness check on peer0.org1.

```
kali@kali: ~/fabric-dev/fabric-samples/test-network  
Session Actions Edit View Help  
"Org2MSP": false  
}  
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'  
Using organization 2  
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...  
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.  
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json  
+ res=0  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": false  
  }  
}  
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'  
Using organization 2  
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile  
/home/kali/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-  
cert.pem --channelID mychannel --name basic --version 1.0 --package-id basic_1.0:1d4d445573112813889f87c307bc2b5fbc664c87b2  
4c035bee15cbcc7f59b629 --sequence 1  
+ res=0  
2026-09-08 13:30:31.964 EDT 0001 INFO [chaincodeCmd] ClientWait -> txid [3695ba7a0f1736f35115c3877e8a71b5bbddc5361e541ecd1e  
fd8af4010c3ef3] committed with status (VALID) at localhost:9051  
Chaincode definition approved on peer0.org2 on channel 'mychannel'  
Using organization 1  
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'mychannel'...  
Attempting to check the commit readiness of the chaincode definition on peer0.org1, Retry after 3 seconds.  
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json  
+ res=0  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}  
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'  
Using organization 2  
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...  
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.  
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json  
+ res=0  
{  
  "approvals": {  
    "Org1MSP": true,  
    "Org2MSP": true  
  }  
}
```

Figure 1: Chaincode approval, commit readiness check, and commitment on mychannel

Step 5: Initialize the Ledger

```
peer chaincode invoke ... -c '{"function":"InitLedger","Args":[]}'
```

Step 6: Query All Assets

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

Step 7: Stop the Network

```
./network.sh down
```

Step 8: Restart and Deploy with AND Endorsement Policy

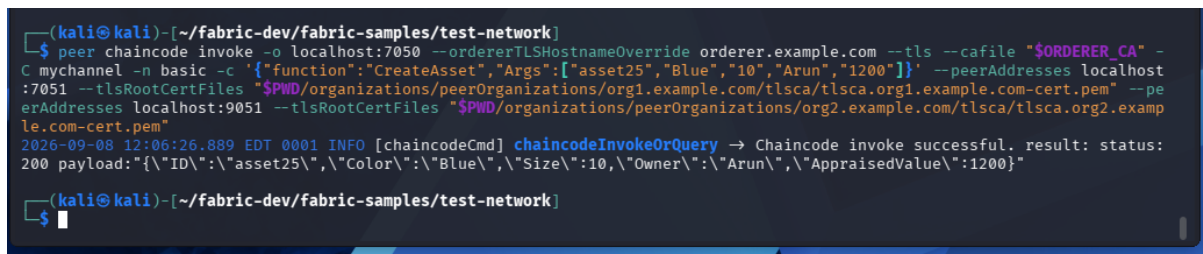
Restart the network and deploy the chaincode requiring both organizations to endorse.

```
./network.sh up createChannel -ca
./network.sh deployCC -ccl javascript -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

Step 9: Submit a Transaction (CreateAsset)

Create asset25 — transaction requires endorsement from both Org1 and Org2.

```
peer chaincode invoke ... \
  -C
  '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}'
```



```
(kali@kali)~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c '{"function":"CreateAsset","Args":["asset25","Blue","10","Arun","1200"]}' --peerAddresses localhost:7051 --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem" --peerAddresses localhost:9051 --tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem"
2026-09-08 12:06:26.889 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:
200 payload:{"ID":"asset25","Color":"Blue","Size":10,"Owner":"Arun","AppraisedValue":1200}
```

Figure 2: CreateAsset transaction endorsed successfully by both organizations

Step 10: Verify the Asset

```
peer chaincode query -C mychannel -n basic -c
  '{"Args":["ReadAsset","asset25"]}'
```



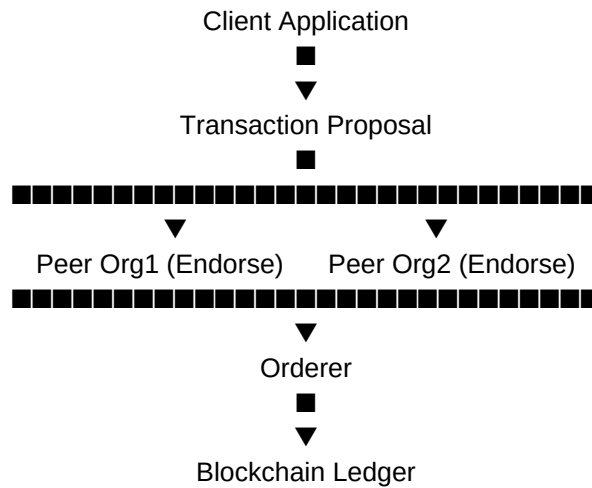
```
(kali@kali)~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset25"]}'
{"AppraisedValue":1200,"Color":"Blue","ID":"asset25","Owner":"Arun","Size":10}
```

Figure 3: ReadAsset confirms asset25 was created with the correct attributes

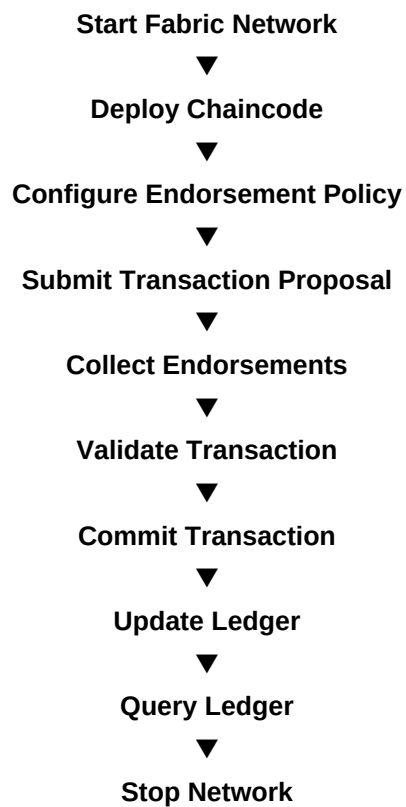
Step 11: Stop the Network

```
./network.sh down
```

Architecture Diagram



Flow Diagram



Result

The endorsement policies were successfully configured in the Hyperledger Fabric network. Transactions were validated according to the specified endorsement rules, demonstrating how Hyperledger Fabric ensures trust, integrity, and secure transaction processing through endorsement policies.